Table of Contents

I. System Architecture *2*

1. Package Diagram **2**

2. Layer descriptions **3**

II. Use Case Model (with Cross-Referenced Diagrams with the Use Cases) *6*

III. Design Pattern Implementations *16*

UML Class Diagrams **16**

V. Generative AI Disclosure *18*

VI. Contribution Self-Declaration *19*

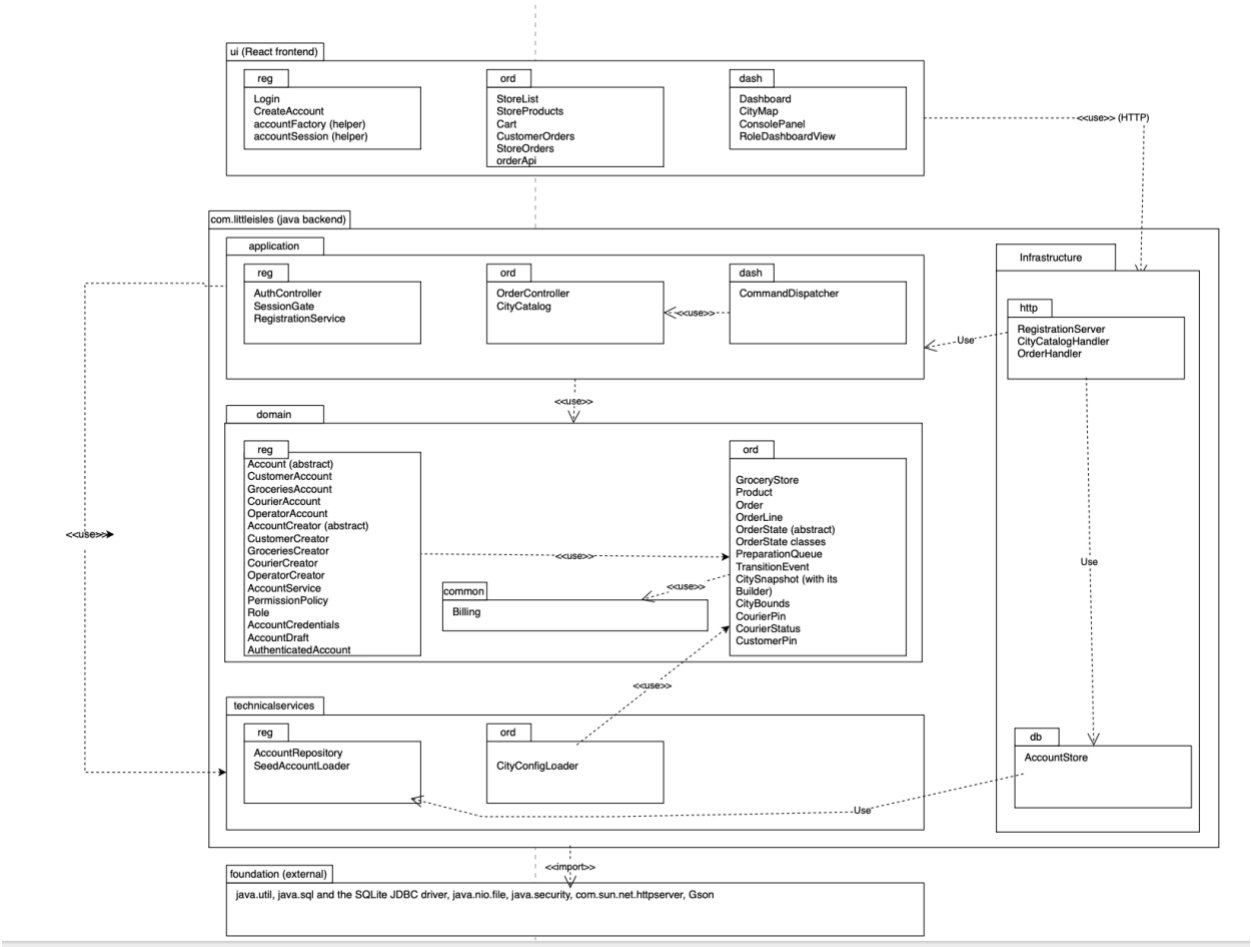
I. System Architecture

For our project, we are implementing a layered architecture, as it is one of the simplest and lowest-cost architectural styles, which suits a project with a tight time constraint like ours. It separates concerns: each layer is responsible for its own part of the system, so changes made in one layer do not impact the components of the others. At the start of the project, we discussed a domain-partitioned architecture such as microservices, because some of our members had worked with it in previous projects and internships. However, the course material showed us it was not the best option: it implies duplicating logic and data across services, and its cost and complexity are not time-efficient for a project of our size.

The system is deployed as two units: the React frontend, which runs in the user's browser, and the Java backend, which contains the application, domain and technical services layers and stores its data in an embedded SQLite database file. Our layers are closed, meaning each layer only uses the layer directly below it, except for three known cases described in sections 2.4 and 2.5.

1. Package Diagram

Figure 1.2 Little Isles Package diagram



2. Layer descriptions

1.1. UI layer

The UI layer is the presentation layer: it handles all user interface logic and rendering. It forms the ui package of the React frontend. The reg sub-package holds the **Login** and **CreateAccount** pages. The ord sub-package holds the ordering screens: **StoreList** shows the stores loaded from the city configuration with their status (R-ORD-01), **StoreProducts** shows a store's products and whether they are available, **Cart** lets the customer review and place an order, **CustomerOrders** lets a customer follow their orders, and **StoreOrders** lets store staff accept or reject pending

orders and work through the preparation queue. **orderApi** is the only file that sends order requests to the backend. The dash sub-package holds the **Dashboard**, which shows each role its own view, the **CityMap**, and the **ConsolePanel**, which logs every command and error. The UI layer contains no business rules; it sends every request over HTTP to the backend's infrastructure.http package.

1.2. Application layer

The application layer is part of the business layer. It coordinates the system's workflows in response to requests from the UI. The reg sub-package holds **AuthController**, which handles login, logout and sessions, **SessionGate**, which verifies that each request has a valid session (R-REG-02), and **RegistrationService**, which creates accounts. The ord sub-package holds **OrderController**, which places orders and handles the store's accept, reject and ready commands, and **CityCatalog**, which holds the city loaded at startup. The dash sub-package holds **CommandDispatcher**, the single-entry point for order commands: it checks the caller's role before passing each command to **OrderController**.

1.3.Domain layer

The domain layer holds the domain objects and enforces the business rules. The reg sub-package holds the abstract **Account** and one subclass per role (**CustomerAccount**, **GroceriesAccount**, **CourierAccount**, **OperatorAccount**), created through **AccountCreator** and its subclasses, which apply the Factory Method pattern so that every account has exactly one role (R-REG-01). **AccountService**, **PermissionPolicy** and **Role** define what each role may do. The ord sub-package holds **GroceryStore**, **Product**, **Order** and **OrderLine**. An order's lifecycle is managed by **OrderState** and its subclasses, which apply the State pattern so that an order can only move through permitted transitions (R-ORD-04). **PreparationQueue** keeps each store's orders in arrival order (R-ORD-03), and **TransitionEvent** records every state change (R-ORD-05). **CitySnapshot** holds the loaded city and is assembled with the Builder pattern. The reg sub-package depends on ord, because a customer account holds its orders and a groceries account is linked to its store. Finally, common holds **Billing**, a Singleton for shared pricing rules.

1.4.Technical services layer

The technical services layer handles loading and persistence. The reg sub-package holds the **AccountRepository** interface and **SeedAccountLoader**, which loads the demonstration accounts from a configuration file (R-REG-03). The ord sub-package holds **CityConfigLoader**, which reads the initial city configuration and builds the **CitySnapshot** (R-ORD-01). In this phase, orders are kept in memory. This layer deviates from strict layering in two ways. First, **SeedAccountLoader** calls **RegistrationService** in the application layer, a layer above it, so that seeded accounts are created through the same factory as registered ones. Second, the loaders create domain objects directly when loading the accounts and the city. Both are candidates for refactoring in Phase III.

1.5. Infrastructure

The infrastructure package holds the technical adapters at the edges of the backend. The http sub-package holds **RegistrationServer**, **CityCatalogHandler** and **OrderHandler**, which receive the frontend's HTTP requests, pass them to the application layer, and return JSON responses. The db sub-package holds **AccountStore**, which implements **AccountRepository** using the SQLite database. The HTTP handlers also use domain classes directly to build their responses, rather than going only through the application layer; this keeps the handlers simple, at the cost of coupling them to the domain model.

1.6.Foundation

The foundation consists of the external libraries the layers build on: java.util for collections, java.sql with the SQLite JDBC driver for the database, java.nio.file for reading configuration files, java.security for password hashing, the JDK's built-in HTTP server (com.sun.net.httpserver), and Gson for JSON.

II. Use Case Model (with Cross-Referenced Diagrams with the Use Cases)

ID:	UC-REG-01
Title:	Creating account
Description:	A person can create an account with exactly one assigned role: a customer, a restaurant, a courier and an operator
Primary Actor:	Customer
Preconditions:	The system is running and the does not already have an account with the same credentials
Postconditions:	A new account is created and saved in the system
Inputs:	Account credentials, user information and account role
Outputs:	A confirmation for the account creation and the assigned account role
Main Success Scenario:	1. The user request to create an account 2. The system display the account creation form 3. The user enters credentials for his new account 4. The system validates information and ensures exactly one role is assigned to the new account 5. The system creates and saves the account 6. The system displays a confirmation message

ID:	UC-REG-02
Title:	Login
Description:	A user logs into the system using their credentials and can have access to the software depending on the functionalities permitted by their role.
Primary Actor:	Customer, Grocery Owner/Staff, Courier and/or Operator
Preconditions:	The user has an existing account and is not currently logged in
Postconditions:	The user is authenticated, an active session is established, and the user can access the functionalities permitted by their role
Inputs:	User login credentials
Outputs:	Login confirmation, authenticated session and access to the system
Main Success Scenario:	1.The user opens the login page 2.The system displays the login form 3.The user enter his credentials to an existing account 4. The system verifies the credentials 5. The system identifies the user assigned role 6. The system establishes an authenticated session 7. The user is granted access to the functionalities permitted by their role

ID:	UC-REG-03
Title:	Load seeded accounts
Description:	The system loads predefined user accounts from a configuration file to ensure that accounts for all four roles are available for demonstration.
Primary Actor:	System Administrator
Preconditions:	The configuration file exists and contains at least one predefined account for each role
Postconditions:	The predefined accounts are loaded in the system and are available for login
Inputs:	Configuration file containing predefined account information and assigned roles
Outputs:	Loaded user accounts and confirmation for initialization
Main Success Scenario:	1.The system starts initialization 2. The system reads the configuration file 3. The system retrieves the predefined user accounts 4.The system validates that each account has exactly one assigned role 5. The system displays at least one account for each role 6. The account become available for authentication 7. The system completes initialization

ID:	UC-ORD-01
Title:	Load initial city configuration
Description:	The system loads the initial city configuration from a file containing information and makes the data available for display on the map
Primary Actor:	N/A
Preconditions:	The system is being initialized, and the configuration file containing the initial city information is available
Postconditions:	The initial city data is loaded into the system, and the groceries and couriers are displayed on the map according to their locations
Inputs:	Configuration file containing Grocery information: name, status, lat/location, menu; menu items: id, name, price, status; courier information: id, location, status and customer information (id, name, delivery address)
Outputs:	Initialized city data and map displaying the groceries and couriers at their corresponding locations
Main Success Scenario:	The system starts initialization The system reads the city configuration file The system loads the groceries information The system loads the menu items The system loads the courier information The system loads customer information The system initializes the city state using the loaded information The dashboard displays the groceries and couriers on the map according to their locations

ID:	UC-ORD-02
Title:	Place an order
Description:	A customer submits order lines to one grocery. ORD prevents the order if the grocery is paused or closed, or if any item is sold out.
Primary Actor:	Customer
Preconditions:	Customer has an authenticated session with the customer role (UC-REG-02). The grocery exists and has a published menu.
Postconditions:	Success: an order exists in state placed and a transition event id is recorded. Refusal: no order is created and the error is logged to the console.
Inputs:	Session, restaurant id, order lines (menu item id, quantity).
Outputs:	Order id, state placed, event id or a refusal message naming the reason.
Main Success Scenario:	Customer selects a grocery and adds items with quantities. Customer submits the order. System verifies the session and the customer role. System verifies the restaurant is open. System verifies every item is available and every quantity is positive. system creates the order in state placed and records the transition (UC-ORD-05). System logs the command and displays the confirmation.

ID:	UC-ORD-03
Title:	Prepare accepted orders in arrival order

Description:	Grocery staff start preparing orders from the grocery's preparation queue. The queue is first in, first out and sets only the sequence of preparation. It never refuses an order for the lack of capacity.
Primary Actor:	Grocery Owner/Staff
Preconditions:	Actor has an authenticated session with the restaurant role. The grocery's queue holds at least one order in state accepted, ordered by the time each was accepted.
Postconditions:	The order at the head of the queue is state preparing and has left the queue. The remaining orders keep their relative order. A transition event id is recorded.
Inputs:	Session, grocery id (the head order is selected by the system).
Outputs:	Updated order state, event id, new queue head.
Main Success Scenario:	Staff chooses to start the next preparation. System verifies the session and the grocery role. System identifies the order at the head of this grocery's queue. System validates the accepted → preparing transition (UC-ORD-04). System removes the order from the queue and sets it to preparing. System records the transition (UC-ORD-05). System logs the command and updates the display.

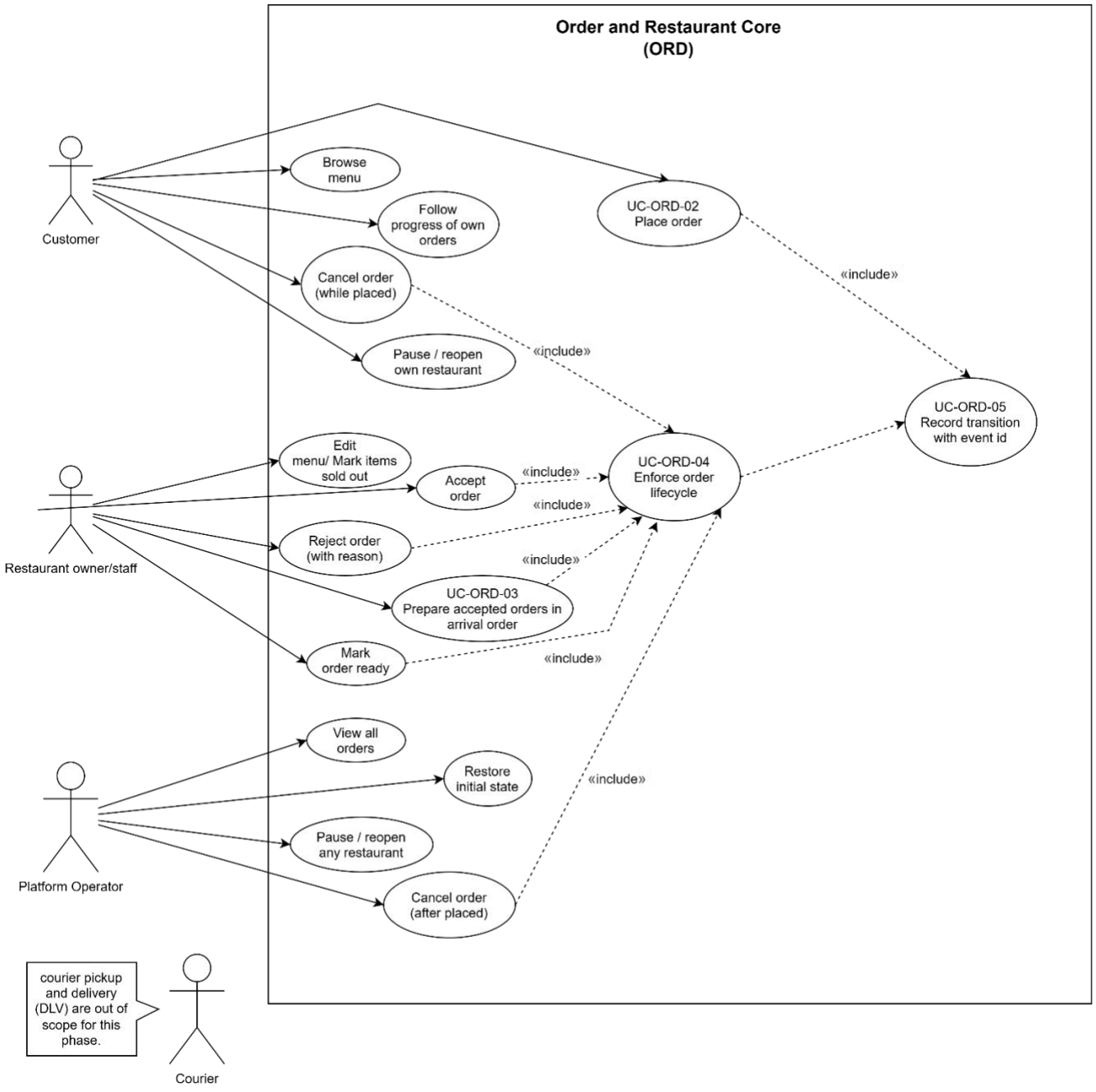
ID:	UC-ORD-04
Title:	Enforce the order lifecycle
Description:	Whenever a command asks to change an order's state, ORD checks the transition against the permitted lifecycle and rejects any transition that skips a state.

Primary Actor:	System (triggered by transition command from a Customer, Grocery Owner/Staff, or Platform Operator)
Preconditions:	A command with a valid session has requested a state change on an existing order.
Postconditions:	Allowed: the order is in the requested next state. Rejected: the order keeps its current state and the error is logged.
Inputs:	Order id, current state, requested target state.
Outputs:	Accepted or rejected decision, with an error message when rejected.
Main Success Scenario:	System receives a transition request for an order. System reads the order's current state. System checks that the target state is a permitted next state: placed → accepted, rejected, or cancelled. Accepted → preparing. Preparing → ready. Ready → picked up. System allows the transition and applies it. System records the transition (UC-ORD-05).

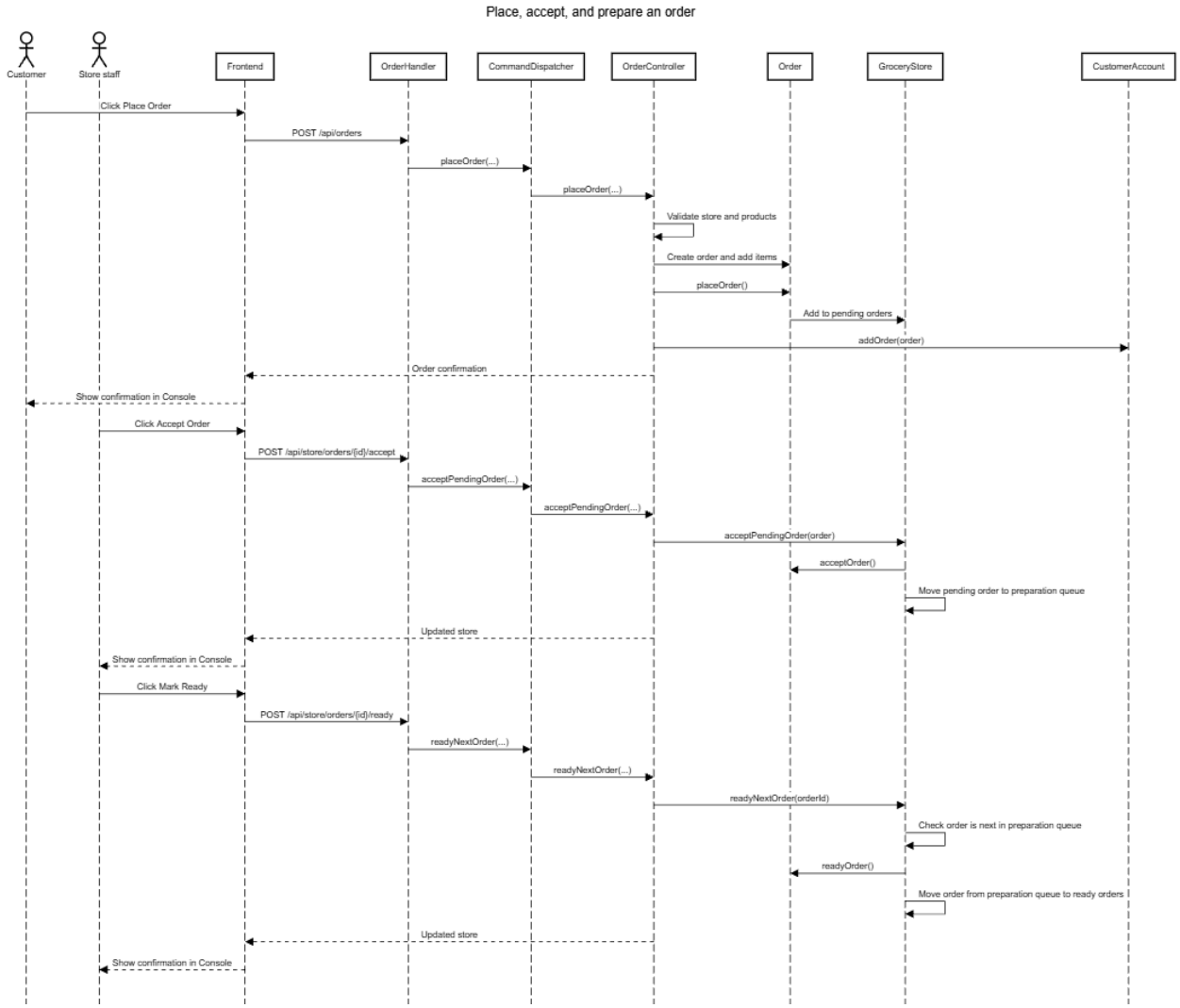
ID:	UC-ORD-05
Title:	Record a state transition with an event id
Description:	Every time an order changes state, ORD stores a transition event with a unique event id, so the history of each order can be traced.
Primary Actor:	System (triggered by any successful state transition)
Preconditions:	A transition has been validated by UC-ORD-04

Postconditions:	A transition event exists with a unique event id, the order id, the previous and new states, the acting user, and a timestamp. The order's state matches the last event recorded for it.
Inputs:	Order id, previous state, new state, acting user, time.
Outputs:	Event id, returned to the calling use case and logged to the console.
Main Success Scenario:	System receives a validated transition. System generates a new, unique event id. System stores the event (order id, previous state, new state, actor, timestamp). System applies the state change to the order. System returns the event id to the caller and logs it to the console.

Use case diagram:



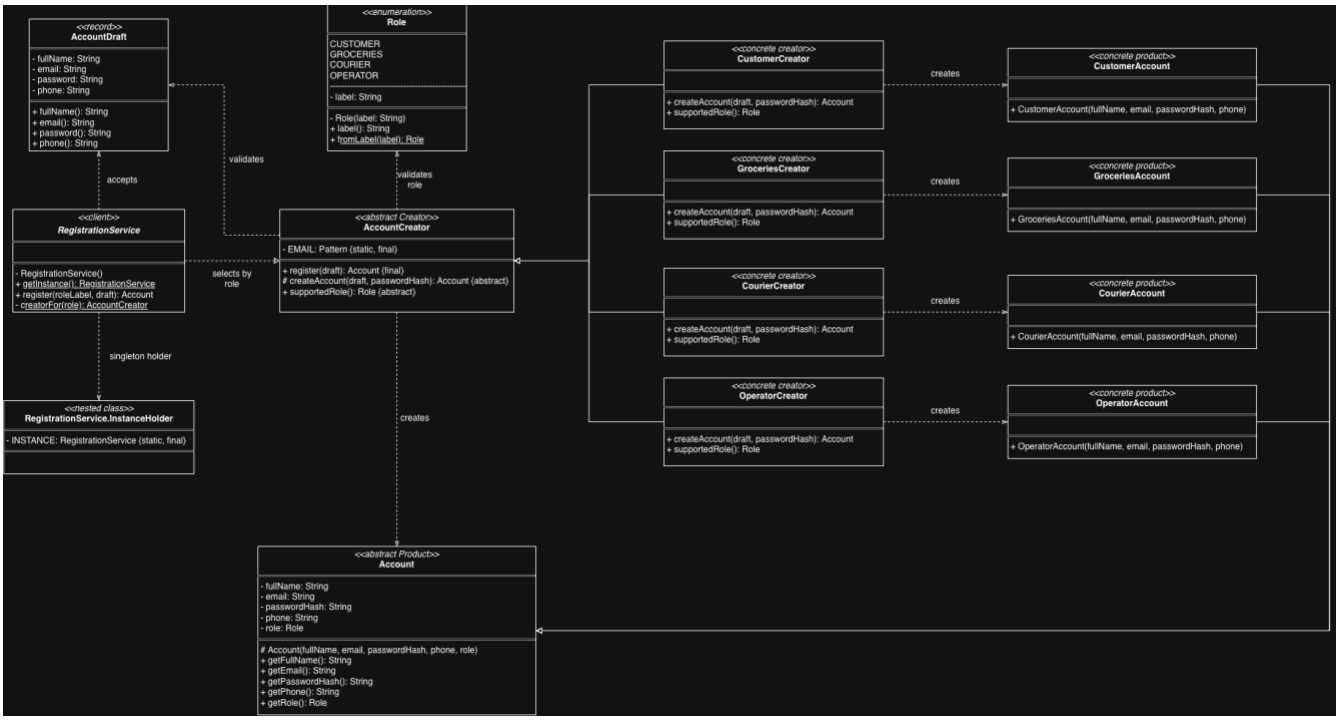
Sequence Diagram:



III. Design Pattern Implementations

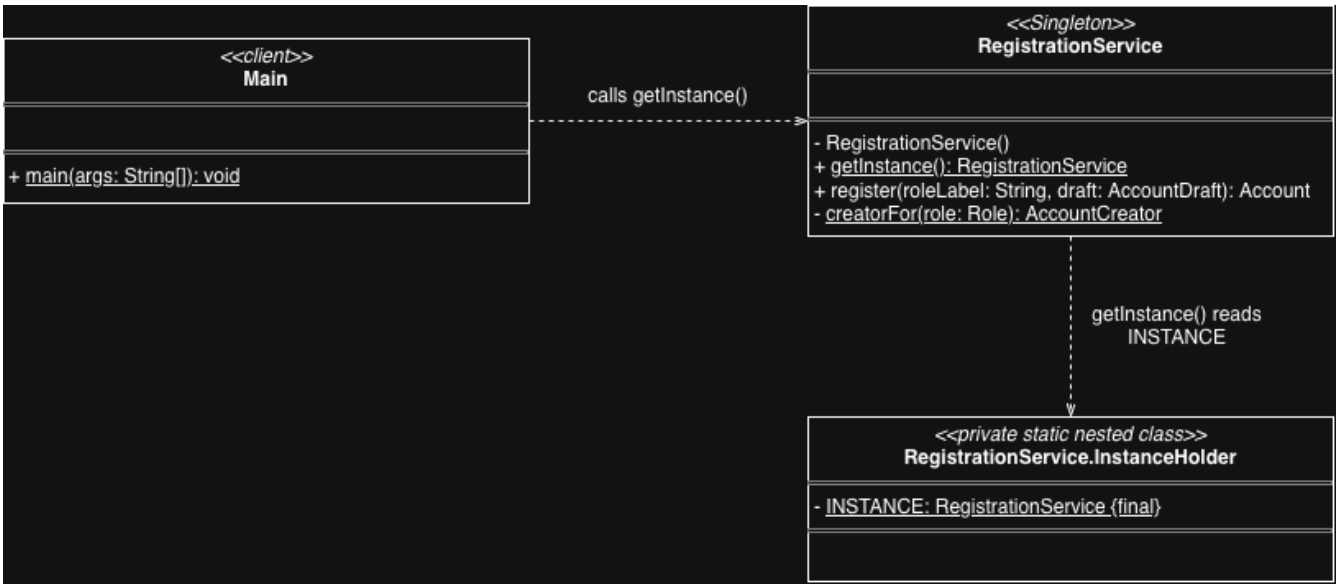
UML Class Diagrams

Factory Method



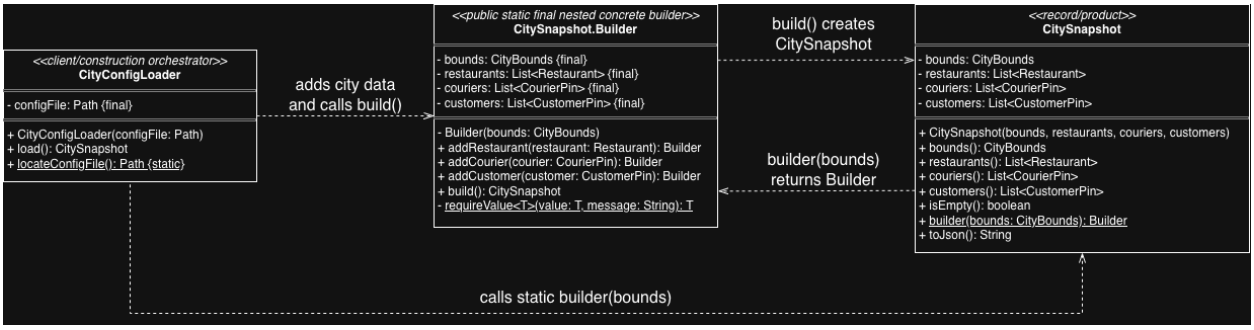
The factory method design pattern was chosen since most account attributes and methods are almost identical to each other, and so that most of the account creation code can be reused to create accounts that will have different attributes. This way, if there's a sweeping change to the structure of all accounts, it takes minimal code to be reflected in all subclasses. New types of accounts would also be trivial to implement.

Singleton



The singleton was implemented in many classes that need only be instantiated once. To ensure its data remains consistent, any attempts to create the object again will simply return the already created instance. While they could have been turned into static classes, a Singleton is a very easy design pattern to implement, and this method should save on resources from no global variables and lazy initialization.

Builder



The builder design pattern was chosen since a city snapshot contains several different parts, such as its bounds, grocery stores, couriers, and customers. The builder allows these parts to be added step by step as they are loaded, making the creation code easier to read and understand. This way, cities with different numbers of stores, couriers, and customers can be created using the same process. Once everything has been added, calling build() creates the finished snapshot.

V. Generative AI Disclosure

Gemini, Claude, ChatGPT (+ Codex)

Team DevOnBevs recognizes the proper use of generative AI tools that were used during the project to aid in the implementation of the code. These tools were utilized to assist with syntax correction, debugging, verify architectural guidelines, verify requirement guidelines from the frontend to the backend. Our team discussed the architectural decisions, design patterns, and the final code structure to be implemented for phase II. The AI outputs were properly reviewed and were used here to ensure that all trivial detail have been acknowledged.

Codex

Codex was present for most steps of the implementation, giving feedback, alternatives, and pre-empting errors that weren't found yet, as well as filling in our web dev knowledge gaps.

Claude

I wrote the first drafts of all the software architecture section myself, then I had Claude review and challenge my understanding of layers, folders and the topology of the layered architecture and point to the relevant chapters in the textbook (the fundamentals of software architecture), which I then reworked, and Claude did another pass to edit it and help with better re-wording. The same was done with the package diagram.

VI. Contribution Self-Declaration

Jeremie Beaudoin

- I worked with Brandon to decide on the design patterns we would be using. I then worked on the backend of UC-ORD-04 and 05 at first. Finally, me and Sheryl finished the frontend order functionality, as well as the main logic for the ordering core.

Brandon Falcon Aguilar

- Thanh-Lam and I separated UC-ORD-01 in a 50/50 separation. I implemented the map API to the web application by doing the initial connections with the four user roles. I created the initial classes so that it follows the Builder design so Thanh-Lam can then continue with its proper implementation.
- I have drawn the class diagrams for three design patterns: factory method, singleton and builder.

Sheryl Ingabire

- I was responsible of creating the package diagram and drafting the system architecture section of our document including rationale behind our architectural decision as well as describing our packages organization alongside Younes (50/50).
- I also implemented part of the UI layer of the UC-ORD-02 and UC-ORD-03 that was then complemented by Jeremy.

Yasmine Rabbath

- Thanh-Lam and I split the use case model (5.2). I wrote the first four use case models in 5.2.
- I completed the sequence diagram for a customer placing an order and the restaurant preparing it.

- For the implementation, I built the foundation of the web app and the first use case (UC-REG-01) which creates an account with exactly one assigned role: a customer, a restaurant, a courier and an operator.
- I configured the database for registering different accounts that were created.

Thanh-Lam Tran

- Yasmine and I split the use case model (5.2). I wrote the 4 last use case models, after Yasmine, so from UC-ORD-02 to UC-ORD-05.
- I worked on the use case diagram. For the implementation part, I was responsible for 50% of the UC-ORD-01, load initial city configuration. I worked on the builder design pattern for the city map.

Younes Zakaria

- I assisted Sheryl Ingabire in the drafting of the package diagram, its initial organization before implementation, and architectural decision. (the work was divided evenly, 50/50)
- I implemented the Login feature to allow users to put in their credentials to grant them access to the web application (UC-REG-02) as well as the seeded accounts which is mainly for demonstration purposes (UC-REG-03).